

Customer Churn Prediction Project

Problem Statement

- Customer churn is a major problem for businesses, as losing customers reduces revenue and growth. The goal of this project is to build a machine learning model that can predict whether a customer will leave the bank (churn) or stay, based on their personal and account information. By identifying customers who are likely to churn, companies can take early actions to retain them and improve customer satisfaction.

Import Libraries

```
In [8]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

Load Dataset

```
In [9]: df = pd.read_csv('C:/Users/Awais/Downloads/ML_Lab_mid/Churn_Modelling.csv')
df.sample(5)
```

```
Out[9]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
1153	1154	15669899	Fitts	755	Germany	Female	45	7
1795	1796	15757648	Marshall	683	Germany	Female	35	5
8977	8978	15626628	Tretiakova	631	Spain	Female	31	2
4402	4403	15720637	Bell	710	Germany	Female	46	10
8733	8734	15714241	Haddon	749	Spain	Male	42	9



```
In [10]: df.shape
```

```
Out[10]: (10000, 14)
```

```
In [11]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   RowNumber             10000 non-null  int64
 1   CustomerId            10000 non-null  int64
 2   Surname               10000 non-null  object
 3   CreditScore           10000 non-null  int64
 4   Geography             10000 non-null  object
 5   Gender               10000 non-null  object
 6   Age                  10000 non-null  int64
 7   Tenure               10000 non-null  int64
 8   Balance              10000 non-null  float64
 9   NumOfProducts        10000 non-null  int64
10   HasCrCard            10000 non-null  int64
11   IsActiveMember       10000 non-null  int64
12   EstimatedSalary      10000 non-null  float64
13   Exited               10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
In [12]: df.describe()
```

```
Out[12]:
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.00000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.88928
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.40520
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.00000
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.00000
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.54000
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.24000
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.09000



Check_Missing_Values

```
In [13]: df.isnull().sum()
```

```
Out[13]: RowNumber      0
         CustomerId     0
         Surname        0
         CreditScore     0
         Geography      0
         Gender         0
         Age            0
         Tenure         0
         Balance        0
         NumOfProducts  0
         HasCrCard      0
         IsActiveMember 0
         EstimatedSalary 0
         Exited         0
         dtype: int64
```

Drop Irrelevant Columns

```
In [14]: df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1, inplace=True)
```

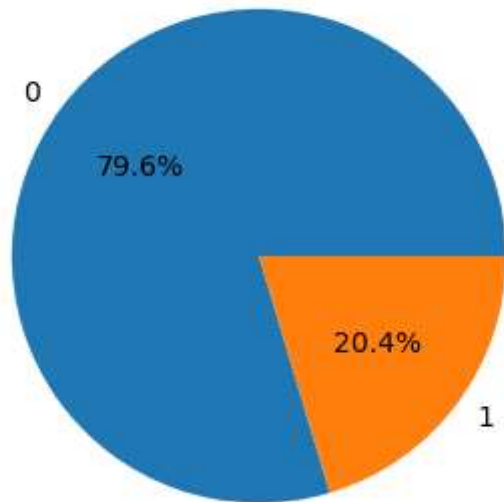
- Removed unnecessary columns that do not help prediction.

Exploratory_Data_Analysis

Target Variable Distribution

```
In [15]: plt.figure(figsize=(4,4))
         df['Exited'].value_counts().plot.pie(autopct='%1.1f%%')
         plt.title("Churn Distribution")
         plt.ylabel("")
         plt.show()
```

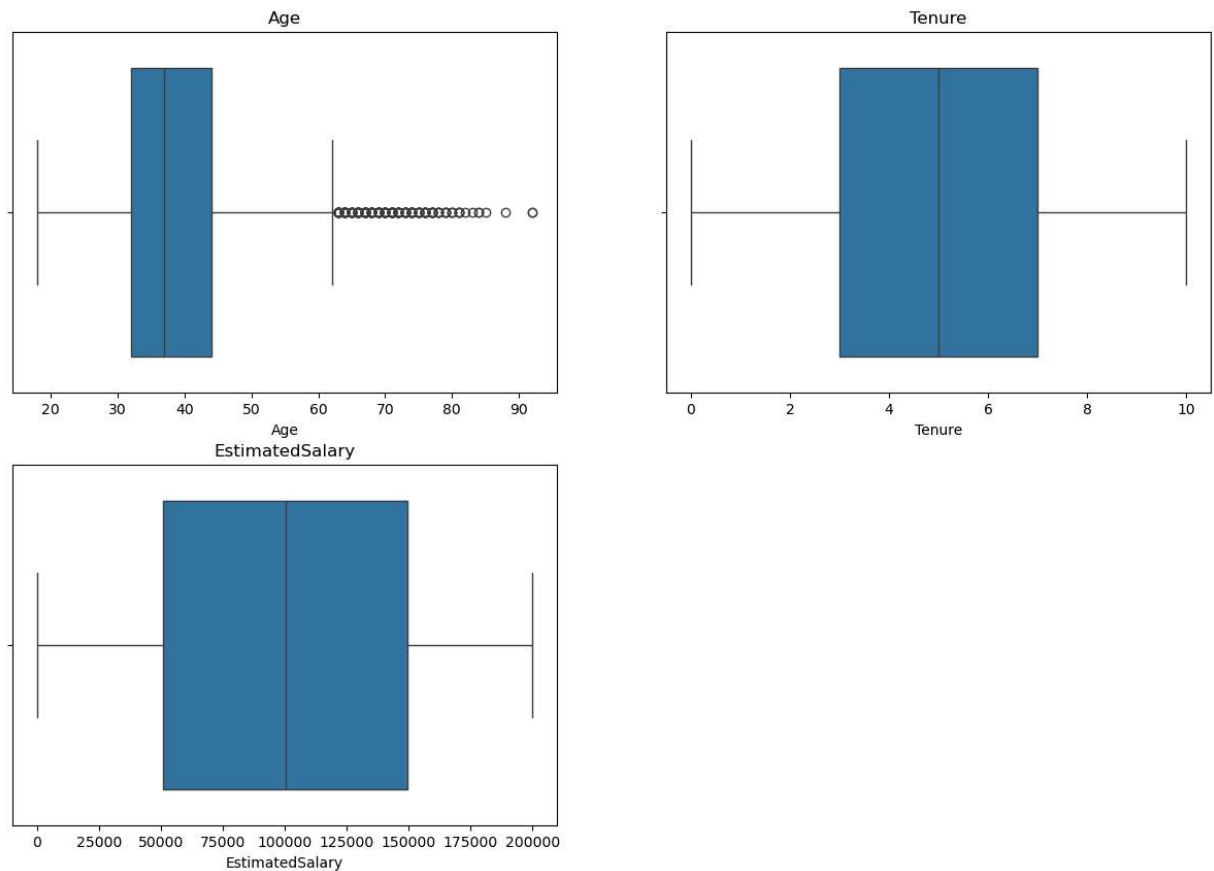
Churn Distribution



- The target columns looks imbalanced

Numerical Features

```
In [16]: plt.figure(figsize=(15,10))
for i, col in enumerate(['Age', 'Tenure', 'EstimatedSalary']):
    plt.subplot(2,2,i+1)
    sns.boxplot(x=df[col])
    plt.title(col)
plt.show()
```



- It does not look like there are many outliers in the numerical features, except Age.

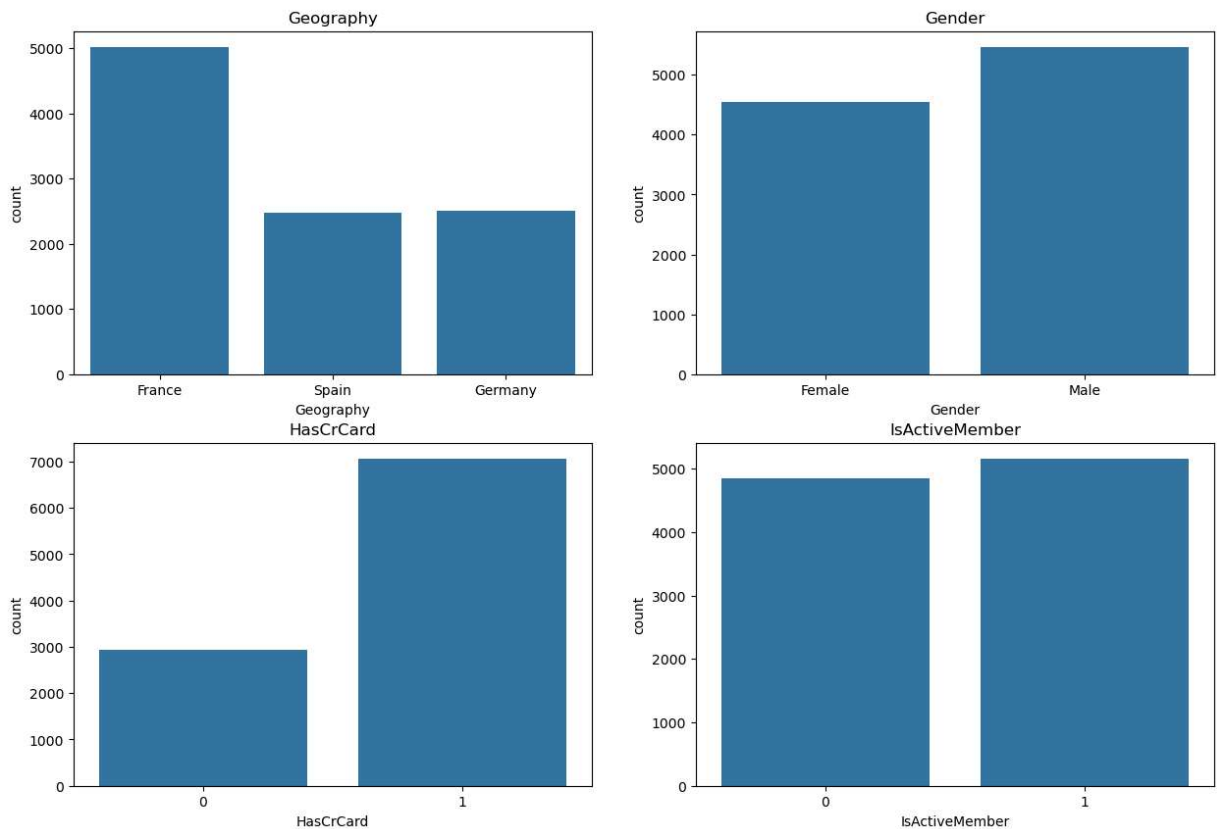
```
In [17]: # Outlier treatment using IQR method
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1
```

```
In [18]: lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df['Age'] = np.clip(df['Age'], lower_bound, upper_bound)
print(f"Age range after fixing: {df['Age'].min()} to {df['Age'].max()}")
```

Age range after fixing: 18 to 62

Categorical Features

```
In [19]: plt.figure(figsize=(15,10))
for i, col in enumerate(['Geography', 'Gender', 'HasCrCard', 'IsActiveMember']):
    plt.subplot(2,2,i+1)
    sns.countplot(x=df[col])
    plt.title(col)
plt.show()
```



- The count plots indicate that there are more customers from France than Spain or Germany, Gender almost balanced, Most customers have credit cards, Active members are slightly higher

Churn

```
In [20]: print("Observing independent variables based on the Exited Column:")

fig, axes = plt.subplots(3,2, figsize=(24,20))

sns.boxplot(data=df, y='CreditScore', x='Exited', ax=axes[0,0])
axes[0,0].set_title('Credit Score Distribution by Churn Status')

sns.boxplot(data=df, y='Age', x='Exited', ax=axes[0,1])
axes[0,1].set_title('Age Distribution by Churn Status')
axes[0,1].tick_params(axis='x', rotation=45)

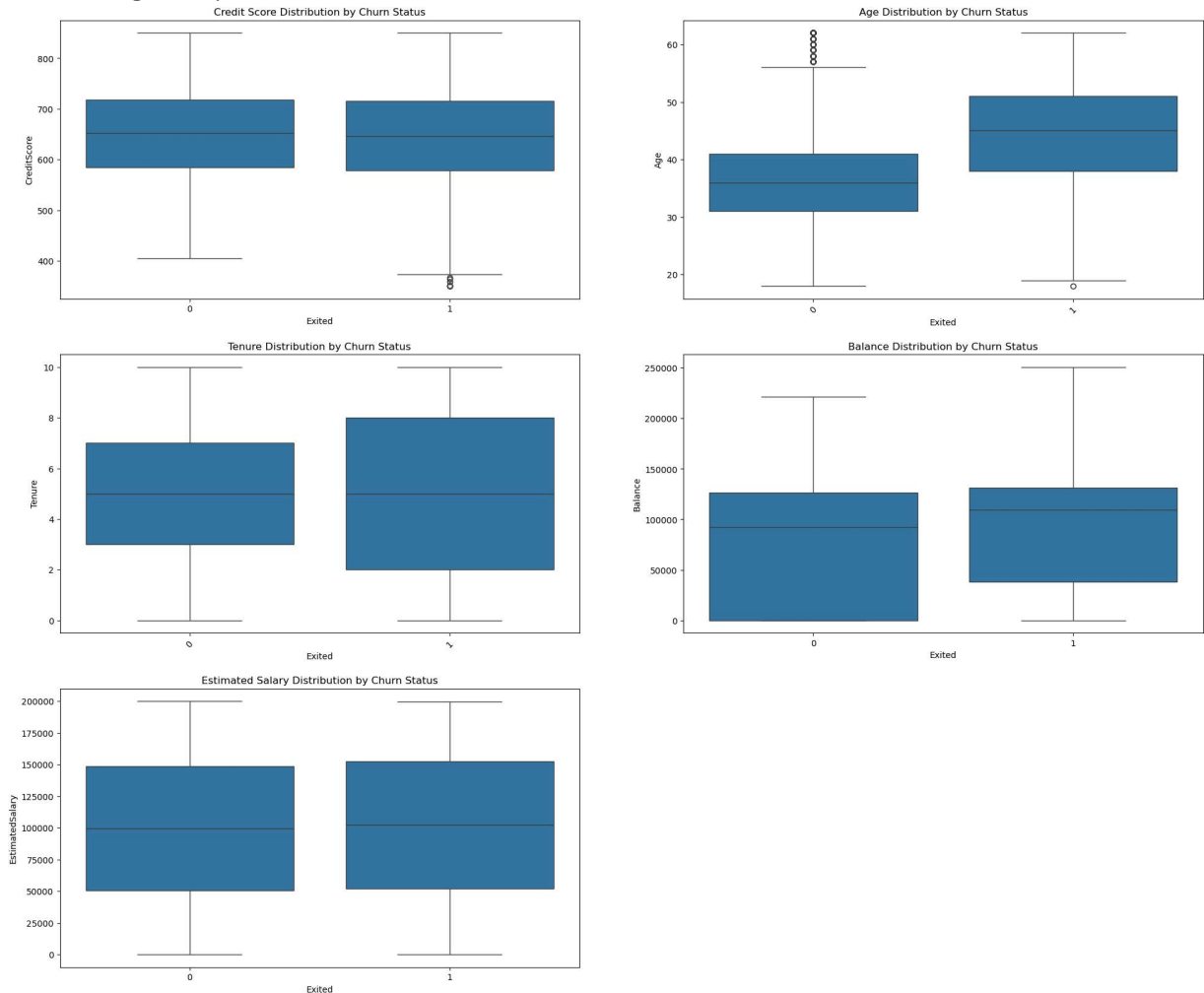
sns.boxplot(data=df, y='Tenure', x='Exited', ax=axes[1,0])
axes[1,0].set_title('Tenure Distribution by Churn Status')
axes[1,0].tick_params(axis='x', rotation=45)

sns.boxplot(data=df, y='Balance', x='Exited', ax=axes[1,1])
axes[1,1].set_title('Balance Distribution by Churn Status')

sns.boxplot(data=df, y='EstimatedSalary', x='Exited', ax=axes[2,0])
axes[2,0].set_title('Estimated Salary Distribution by Churn Status')
```

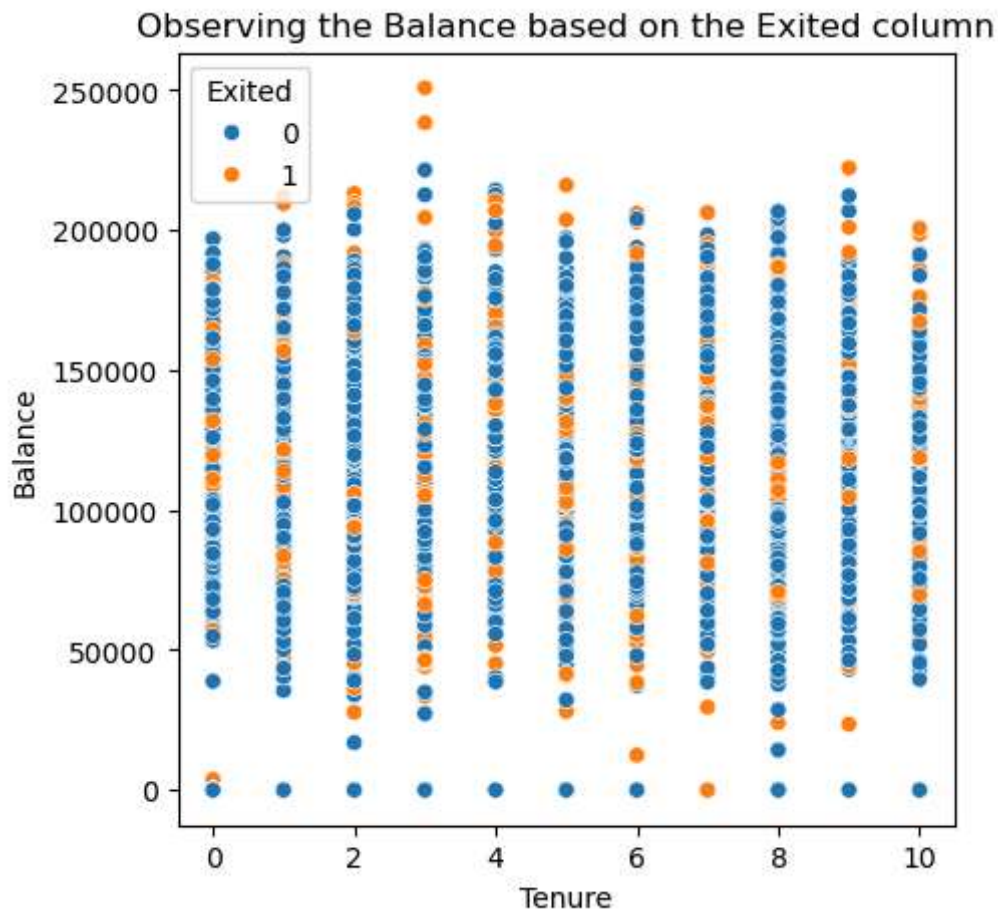
```
axes[2,1].axis('off')
plt.show()
```

Observing independent variables based on the Exited Column:



- The credit score distribution does not show a stark difference, indicating that the credit score may not be a strong predictor of churn.
- The tenure distribution suggests that customers with a shorter tenure are slightly more likely to churn than those with a longer tenure.
- Age distribution shows older customers appear more likely to churn than younger ones.
- The balance distribution is significantly different, so Customers with higher balance tend to churn more.
- The estimated salary distribution does not show a stark difference, so salary may not be a primary factor in the decision to churn.

```
In [21]: plt.figure(figsize=(5,5))
sns.scatterplot(x='Tenure', y='Balance', hue='Exited', data=df)
plt.title('Observing the Balance based on the Exited column')
plt.show()
```

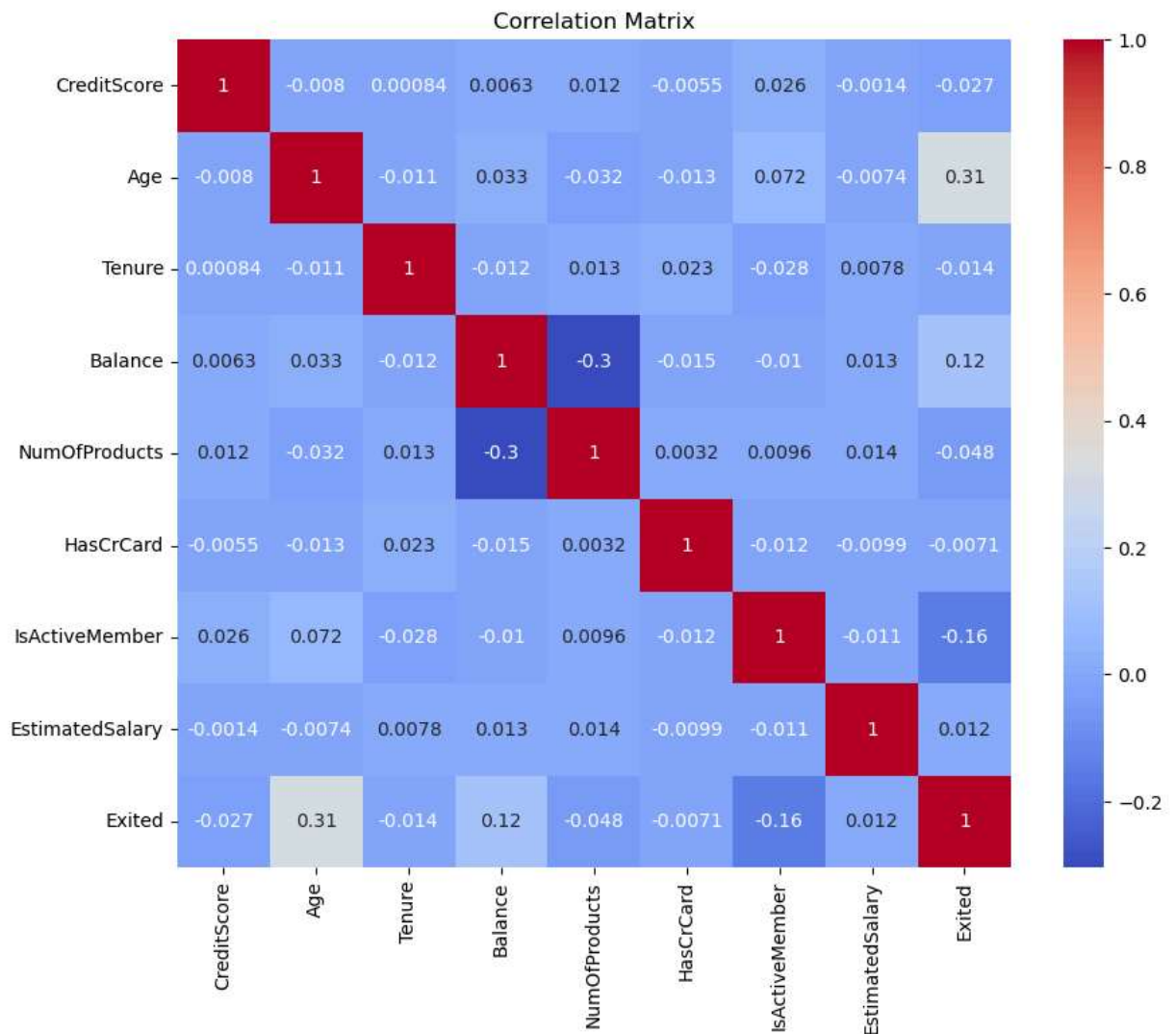


- scatter plot indicates that there is no clear pattern or correlation between tenure and balance for both churned and retained customers, implying these factors independently do not strongly predict customer churn

Correlation Heatmap

```
In [22]: num_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(10,8))
sns.heatmap(num_df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

- Age positively correlated with churn, Active members less likely to churn and Balance has moderate effect

Data Preprocessing

Encoding

```
In [23]: df = pd.get_dummies(df, columns=['Geography', 'Gender'], drop_first=True)
df.head()
```

Out[23]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	Esti
0	619	42	2	0.00	1	1	1	
1	608	41	1	83807.86	1	0	1	
2	502	42	8	159660.80	3	1	0	
3	699	39	1	0.00	2	0	0	
4	850	43	2	125510.82	1	1	1	



- Converted categorical data into numbers for model training.

Split Data

```
In [24]: X = df.drop('Exited', axis=1)
y = df['Exited']
```

```
In [25]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Scaling

```
In [26]: scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

- Scaled data so all features have equal importance.

Model_Implementation -> Before Tuning

- **Logistic Regression**

```
In [27]: lr = LogisticRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

- Trains baseline logistic regression model.
- **KNN**

```
In [28]: knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
```

- **SVM**

```
In [29]: svm = SVC(kernel='rbf')
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
```

- **Decision Tree**

```
In [30]: dt = DecisionTreeClassifier(max_depth=3)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
```

- **Neural Network**

```
In [31]: nn = MLPClassifier(hidden_layer_sizes=(50,))
nn.fit(X_train, y_train)
y_pred_nn = nn.predict(X_test)
```

```
c:\Users\Awais\anaconda3\Lib\site-packages\sklearn\normalization\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
warnings.warn(
```

Manual Tuning -> After Changes

Logistic_Regression_Tuning

```
In [32]: lr = LogisticRegression(C=0.5, max_iter=200)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

- Changed regularization and class weight to improve performance.

KNN_Tuning

```
In [33]: knn2 = KNeighborsClassifier(n_neighbors=7)
knn2.fit(X_train, y_train)
y_pred_knn2 = knn2.predict(X_test)
```

- Increased K to improve generalization.

Decision_Tree_Tuning

```
In [34]: dt2 = DecisionTreeClassifier(max_depth=6)
dt2.fit(X_train, y_train)
y_pred_dt2 = dt2.predict(X_test)
```

- Increased depth for better learning.

SVM_Tuning

```
In [35]: svm2 = SVC(kernel='rbf', C=10, gamma=0.01)
svm2.fit(X_train, y_train)
y_pred_svm2 = svm2.predict(X_test)
```

- Tuned hyperparameters for better accuracy.

Neural Network

```
In [42]: nn2 = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500)
nn2.fit(X_train, y_train)
y_pred_nn2 = nn2.predict(X_test)
```

- Threshold Adjustment

```
In [43]: y_pred_nn2_proba = nn2.predict_proba(X_test)[:, 1]
threshold = 0.35
y_pred_nn2_threshold = (y_pred_nn2_proba >= threshold).astype(int)
```

- Increased neurons and iterations.

Evaluation Function

```
In [44]: def evaluate(y_test, y_pred):
    return [
        accuracy_score(y_test, y_pred),
        precision_score(y_test, y_pred, average='weighted'),
        recall_score(y_test, y_pred, average='weighted'),
        f1_score(y_test, y_pred, average='weighted')
    ]
```

- Calculates all required metrics.

Performance Comparison

```
In [45]: results = pd.DataFrame({
    'Model': ['LR', 'KNN', 'KNN Tuned', 'SVM', 'SVM Tuned', 'DT', 'DT Tuned', 'NN',
    'Scores': [
        evaluate(y_test, y_pred_lr),
        evaluate(y_test, y_pred_knn),
        evaluate(y_test, y_pred_knn2),
        evaluate(y_test, y_pred_svm),
        evaluate(y_test, y_pred_svm2),
        evaluate(y_test, y_pred_dt),
        evaluate(y_test, y_pred_dt2),
        evaluate(y_test, y_pred_nn),
        evaluate(y_test, y_pred_nn2)
    ]
})

results[['Accuracy', 'Precision', 'Recall', 'F1']] = pd.DataFrame(results['Scores'].to
results.drop('Scores', axis=1, inplace=True)

results
```

```
Out[45]:
```

	Model	Accuracy	Precision	Recall	F1
0	LR	0.8165	0.786618	0.8165	0.783620
1	KNN	0.8295	0.815053	0.8295	0.819291
2	KNN Tuned	0.8345	0.815759	0.8345	0.815672
3	SVM	0.8575	0.849023	0.8575	0.837213
4	SVM Tuned	0.8560	0.850452	0.8560	0.831994
5	DT	0.8485	0.834731	0.8485	0.834880
6	DT Tuned	0.8590	0.847850	0.8590	0.845516
7	NN	0.8630	0.852746	0.8630	0.851232
8	NN Tuned	0.8610	0.851454	0.8610	0.853157

- The tuned Neural Network achieved the best performance with an F1-score of 0.8531, showing the most balanced results.
- Overall, neural-based models performed better than traditional machine learning models.

Best Model

```
In [46]: # Sort models based on F1 Score
best_model = results.sort_values(by='F1', ascending=False)
print("Best Model:")
print(best_model.head(8))
```

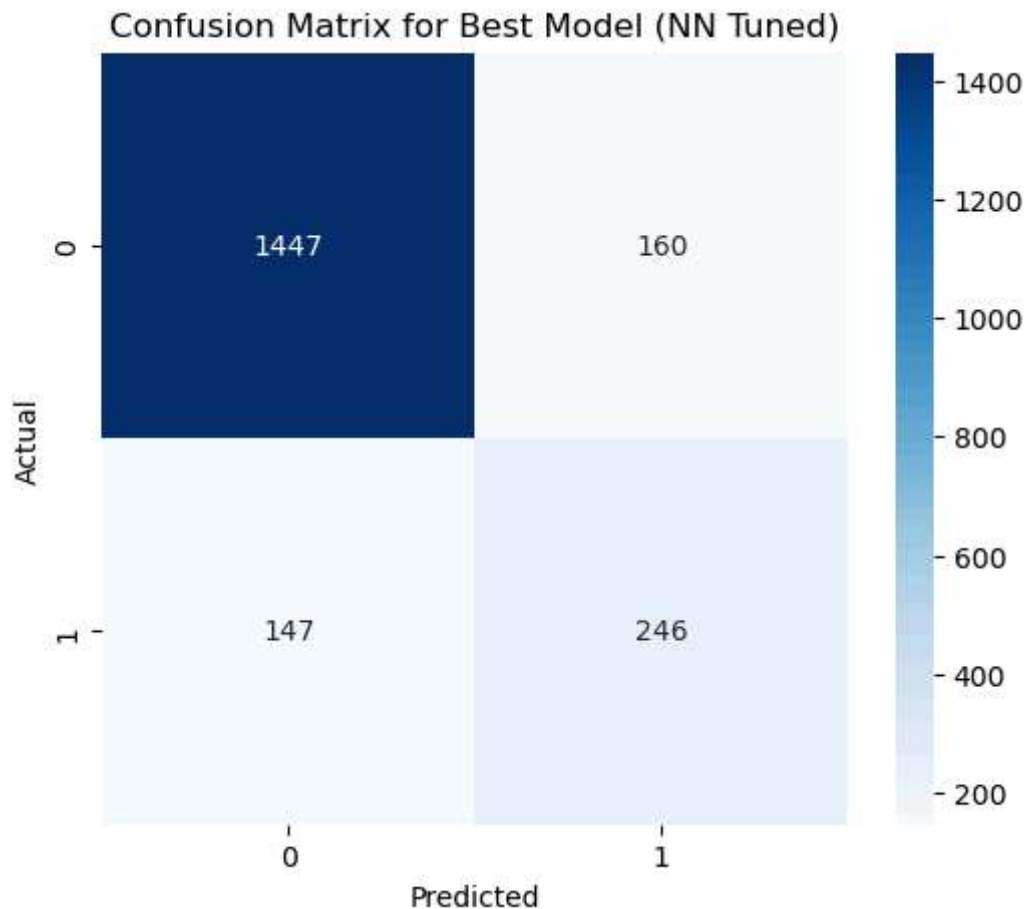
Best Model:

	Model	Accuracy	Precision	Recall	F1
8	NN Tuned	0.8610	0.851454	0.8610	0.853157
7	NN	0.8630	0.852746	0.8630	0.851232
6	DT Tuned	0.8590	0.847850	0.8590	0.845516
3	SVM	0.8575	0.849023	0.8575	0.837213
5	DT	0.8485	0.834731	0.8485	0.834880
4	SVM Tuned	0.8560	0.850452	0.8560	0.831994
1	KNN	0.8295	0.815053	0.8295	0.819291
2	KNN Tuned	0.8345	0.815759	0.8345	0.815672

- After sorting models based on F1-score, the Tuned Neural Network ranked first with Accuracy = 0.8610 and F1-score = 0.8531

Confusion Matrix

```
In [51]: # define confusion matrix for the best model
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_nn2_threshold)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix for Best Model (NN Tuned)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



Interpretation

- True Negatives (1447): The model Correctly predicted non-churn(stay) customers
- True Positives (246): The model Correctly predicted churn customers.
- False Positives (160): The model incorrectly predicted these customers would churn, but they actually stayed.
- False Negatives (147): These are the customers you predicted would stay, but they actually left (important error).

The model performs well in identifying non-churn customers, but it misses some churn cases

.